



CẤU TRÚC DỮ LIỆU & GIẢI THUẬT

BÀI 7: Hashing và tra cứu nhanh

PHẦN 1



Bài toán tra cứu

Tình huống thực tế:

Có danh sách 1 triệu sinh viên.

Muốn tra “sinh viên mã số X có tồn tại không?”

- Cách 1: Duyệt từ đầu đến cuối $\rightarrow O(n)$.
- Cách 2: Dùng hash table $\rightarrow O(1)$ trung bình.

Câu hỏi: tại sao $O(1)$?

Ý tưởng hash table

Hash table = mảng + hàm băm (hash function).

- Hash function chuyển key thành index trong mảng.
- Lưu value tại vị trí index đó.
- Tra cứu: tính lại index $\rightarrow$ lấy value ngay.

Ví dụ:

- key = “SV12345”
- $\text{hash}(\text{“SV12345”}) = 345$
- Lưu data ở array.

Hình minh họa hash table

```
Key → Hash Function → Index → Value
```

```
"Alice" → hash() → 2 → {"name": "Alice", "age": 20}
```

```
"Bob" → hash() → 5 → {"name": "Bob", "age": 22}
```

```
"Carol" → hash() → 1 → {"name": "Carol", "age": 21}
```

```
[0]: None
```

```
[1]: Carol
```

```
[2]: Alice
```

```
[3]: None
```

```
[4]: None
```

```
[5]: Bob
```

```
...
```

Tra “Alice” $\rightarrow \text{hash}(\text{“Alice”}) = 2 \rightarrow \text{trả array} \rightarrow O(1)$.

Hash function

Phương pháp phổ biến: division method (chia lấy dư).

Ví dụ:

- $\text{key} = \text{"Alice"}, \text{table_size} = 10$
- $\text{hash}(\text{"Alice"}) = (65+108+105+99+101) \% 10 = 478 \% 10 = 8$

→ Lưu "Alice" ở index 8.

```
def hash_function(key, table_size):  
    # Chuyển key thành số  
    hash_val = sum(ord(c) for c in key)  
    # Chia lấy dư cho kích thước bảng  
    return hash_val % table_size
```

Vấn đề: collision (va chạm)

Collision: 2 key khác nhau $\rightarrow$ cùng index.

Ví dụ:

- $\text{hash}(\text{"Alice"}) = 8$
- $\text{hash}(\text{"Bob"}) = 8$

Cả 2 muốn chiếm vị trí 8 $\rightarrow$ xung đột!

Giải pháp:

- Chaining (móc xích).
- Open addressing (thăm dò).

Chaining (separate chaining)

Mỗi ô trong mảng là một danh sách liên kết (linked list).

Nếu “Alice” và “Eve” cùng hash = 2:

Thêm “Eve” vào cuối danh sách ở index 2.

Tra cứu “Eve”:

$\text{hash}(\text{"Eve"}) = 2 \rightarrow$ duyệt list ở index 2 $\rightarrow$ tìm thấy.

Độ phức tạp: $O(1 + \alpha)$ với $\alpha = n/m$.

```
Array:  
[0]: None  
[1]: ["Carol"] → None  
[2]: ["Alice"] → ["Eve"] → None  
[5]: ["Bob"] → None
```


Open addressing (thăm dò)

Tất cả key lưu trong chính mảng (không dùng list).

Nếu vị trí bị chiếm $\rightarrow$ tìm vị trí khác theo quy tắc:

- Linear probing: thử $\text{index}+1$, $\text{index}+2$,...
- Quadratic probing: thử $\text{index}+1^2$, $\text{index}+2^2$,...
- Double hashing: dùng hash function thứ 2.

Ưu điểm: tiết kiệm bộ nhớ (không cần pointer).

Nhược điểm: bảng dễ đầy.

So sánh chaining vs open addressing

	Chaining	Open Addressing
Cấu trúc	Mỗi ô là list	Mỗi ô là key/value
Bộ nhớ	Nhiều hơn (pointer)	Ít hơn
Xử lý collision	Thêm vào list	Thăm dò vị trí khác
Khi bảng đầy	Vẫn thêm được	Phải resize



- Chaining: khi collision nhiều.
- Open addressing: khi tiết kiệm RAM.

Load factor (α)

$$\alpha = n / m$$

- n : số phần tử đang lưu.
- m : kích thước bảng.

Nguyên tắc:

- $\alpha < 0.7$: hiệu suất tốt (ít collision).
- $\alpha > 0.9$: nên resize (tăng m , rehash tất cả).

Python dict tự động resize khi $\alpha \approx 0.67$.

Hash table trong Python

Python có sẵn dict (dùng hash table):

```
# Dict là hash table
students = {}
students["SV001"] = "Nguyen Van A"
students["SV002"] = "Tran Thi B"

# Tra cứu O(1)
print(students["SV001"]) # Nguyen Van A

# Kiểm tra key tồn tại
if "SV003" in students:
    print("Tồn tại")
else:
    print("Không tồn tại") # O(1)
```

Code: hash table

Mỗi `self.table[index]` là list các (key, value).

Collision → thêm vào list.

```
class HashTable:
    def __init__(self, size=10):
        self.size = size
        self.table = [[] for _ in range(size)]

    def hash_function(self, key):
        return hash(key) % self.size

    def insert(self, key, value):
        index = self.hash_function(key)
        # Tìm xem key đã tồn tại chưa
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                self.table[index][i] = (key, value)
                return
        # Chưa tồn tại → thêm mới
        self.table[index].append((key, value))
```

Code: search và delete

```
class HashTable:
    # ... (phần trên)

    def search(self, key):
        index = self.hash_function(key)
        for k, v in self.table[index]:
            if k == key:
                return v
        return None # Không tìm thấy

    def delete(self, key):
        index = self.hash_function(key)
        for i, (k, v) in enumerate(self.table[index]):
            if k == key:
                del self.table[index][i]
                return True
        return False # Không có key
```

Test hash table

insert, search, delete đều gần $O(1)$.

Nếu collision ít $\rightarrow$ list ngắn $\rightarrow$ nhanh.

```
ht = HashTable(size=5)
ht.insert("Alice", 20)
ht.insert("Bob", 22)
ht.insert("Carol", 21)
ht.insert("Dave", 23)

print(ht.search("Alice")) # 20
print(ht.search("Bob"))  # 22
print(ht.search("Eve"))  # None

ht.delete("Bob")
print(ht.search("Bob"))  # None
```

Độ phức tạp hash table

Thao tác	Trung bình	Xấu nhất
Insert	$O(1)$	$O(n)$
Search	$O(1)$	$O(n)$
Delete	$O(1)$	$O(n)$

Xấu nhất $O(n)$:

- Tất cả key hash về 1 index $\rightarrow$ list dài n .
- Thực tế hiếm xảy ra với hash function tốt.

Ứng dụng hash table

Các bài toán tra cứu nhanh:

- Kiểm tra phần tử đã xuất hiện (check duplicate).
- Đếm tần suất xuất hiện (frequency count).
- Tìm 2 số có tổng = target (two sum).
- Cache/memoization (lưu kết quả đã tính).
- Database indexing (chỉ mục CSDL).

Ví dụ: đếm tần suất từ

```
def count_words(text):  
    words = text.split()  
    freq = {} # Hash table  
  
    for word in words:  
        if word in freq:  
            freq[word] += 1  
        else:  
            freq[word] = 1  
  
    return freq  
  
text = "hello world hello python world"  
print(count_words(text))  
# {'hello': 2, 'world': 2, 'python': 1}
```

Độ phức tạp: $O(n)$ với n là số từ (mỗi tra cứu $O(1)$).

Ví dụ: Two Sum

Cho mảng nums và target, tìm 2 số có tổng = target:

```
def two_sum(nums, target):  
    seen = {} # Lưu {giá_trị: index}  
  
    for i, num in enumerate(nums):  
        complement = target - num  
        if complement in seen:  
            return [seen[complement], i]  
        seen[num] = i  
  
    return None  
  
print(two_sum([2, 7, 11, 15], 9)) # [0, 1]
```

Độ phức tạp: $O(n)$ thay vì $O(n^2)$ (brute force).

Khi nào dùng hash table?

Dùng khi:

- Cần tra cứu nhanh (key $\rightarrow$ value).
- Kiểm tra sự tồn tại (membership test).
- Đếm tần suất, nhóm dữ liệu.

Không dùng khi:

- Cần duyệt theo thứ tự (dùng sorted list/tree).
- Cần tìm min/max (dùng heap).
- Key không hash được (list, dict).

Hash table vs array vs linked list

	Array	Linked List	Hash Table
Search	$O(n)$	$O(n)$	$O(1)$
Insert	$O(n)$	$O(1)$	$O(1)$
Delete	$O(n)$	$O(1)$	$O(1)$
Thứ tự	Có	Có	Không
Bộ nhớ	Cố định	Linh hoạt	Linh hoạt

Hash table thắng về tốc độ tra cứu.

Rehashing (khi bảng đầy)

Khi $\alpha > \text{ngưỡng } (0.7)$:

1. Tạo bảng mới lớn gấp đôi.
2. Tính lại hash cho mọi key cũ.
3. Chuyển vào bảng mới.

```
def rehash(self):  
    old_table = self.table  
    self.size *= 2  
    self.table = [[] for _ in range(self.size)]  
  
    for bucket in old_table:  
        for key, value in bucket:  
            self.insert(key, value)
```

Chi phí: $O(n)$ nhưng hiếm xảy ra.

Pattern nhận diện bài toán dùng hash

Tín hiệu trong đề bài	Pattern hash	Ví dụ bài
“Tìm 2 phần tử có tổng = X”	Complement lookup (two-sum)	Two Sum, 3Sum
“Đếm số lần xuất hiện” / “tần suất”	Frequency counting với dict	Word frequency, top K elements
“Nhóm theo tính chất giống nhau”	Custom key trong dict	Group anagrams, group shifted strings
“Subarray có tổng/tính chất X”	Prefix sum + hash map	Subarray sum = k, max length subarray
“Kiểm tra đã gặp chưa”	Hash set	Contains duplicate, first unique
“Tra cứu nhanh nhiều lần”	Hash map/set	LRU cache, design HashMap
“Tìm pattern trong text dài”	Rolling hash (Rabin-Karp)	Repeated DNA, find substring

PHẦN 2



Thuật toán Rabin-Karp: tìm pattern bằng rolling hash

Bài toán: Pattern matching (tìm chuỗi con)

Text T rất dài. Pattern P ngắn hơn. Tìm tất cả vị trí xuất hiện của P trong T.

Brute force:

- Thử mọi vị trí i trong T, so khớp P với $T[i..i+m-1]$.
- Độ phức tạp: $O(n \times m)$ với $n = |T|$, $m = |P|$.

Rabin-Karp (1987):

- Sử dụng rolling hash để giảm xuống $O(n)$ trung bình.
- Ý tưởng: So sánh hash thay vì so ký tự.

Ý tưởng rolling hash

Thay vì so từng ký tự, ta:

- Gán mỗi chuỗi một hash số nguyên.
- So sánh $\text{hash}(P)$ với hash của số trong T .
- Nếu hash khác $\rightarrow$ chắc chắn không trùng.
- Nếu hash bằng $\rightarrow$ mới so ký tự để confirm (tránh collision).

Rolling hash: Hash của cửa sổ mới tính từ hash cũ trong $O(1)$.

Tại sao gọi là “rolling”?

Hash của cửa sổ mới được “cuộn” (roll) từ cửa sổ cũ:

- Loại bỏ ký tự đầu (trái).
- Thêm ký tự mới (phải).
- Tất cả trong $O(1)$.

Đây là thuật toán Rabin-Karp (Richard M. Karp & Michael O. Rabin, 1987).

Cửa sổ trượt (sliding window)

Text: “abcdef”

Pattern “bcd” ($m = 3$).

Các cửa sổ:

- “abc” (0..2)
- “bcd” (1..3)
- “cde” (2..4)
- “def” (3..5)

16	12	9	19	11	8
----	----	---	----	----	---



16	12	9	19	11	8
----	----	---	----	----	---



Brute force: hash từng cửa sổ từ đầu $\rightarrow O(n*m)$.

Rolling hash:

“Đẩy” cửa sổ sang phải:

- Bỏ ký tự cũ (trái).
- Thêm ký tự mới (phải).
- Cập nhật hash trong $O(1)$.

Rolling hash

```
def simple_hash(s):  
    # Không dùng trong thực tế, chỉ minh họa  
    return sum(ord(c) for c in s)  
  
text = "abcdef"  
pattern = "bcd"  
m = len(pattern)  
  
# Hash pattern  
hash_p = simple_hash(pattern)  
  
# Hash cửa sổ đầu  
window = text[0:m]  
hash_w = simple_hash(window)
```

Rabin-Karp: Ví dụ cụ thể từng bước

Text: “abcabc”, Pattern: “cab” (tìm vị trí xuất hiện)

base = 256 (số ký tự ASCII), mod = 101 (số nguyên tố)

Bước 1: Tính $\text{hash}(\text{Pattern}) = \text{hash}(\text{“cab”})$

$$\text{hash} = (c \times 256^2 + a \times 256^1 + b \times 256^0) \bmod 101$$

$$= (99 \times 65536 + 97 \times 256 + 98) \bmod 101$$

$$= 6488674 \bmod 101$$

$$= 41$$

Bước 2: Tính hash của số đầu “abc”

$$\text{hash} = (97 \times 65536 + 98 \times 256 + 99) \bmod 101 = 30$$

Bước 3: So sánh: $30 \neq 41 \rightarrow$ không khớp

Bước 4: Rolling sang “bca”

- Loại ‘a’ (97): $\text{hash} = (\text{hash} - 97 \times 256^2) \times 256$
- Thêm ‘a’ (97): $\text{hash} = \text{hash} + 97$
- $\text{hash}(\text{“bca”}) = 82$ (sau mod 101)
- $82 \neq 41 \rightarrow$ không khớp

Bước 5: Rolling sang “cab”

- $\text{hash}(\text{“cab”}) = 41$
- $41 = 41 \rightarrow \text{KHỚP!} \rightarrow \text{Kiểm tra ký tự thực: “cab”} = \text{“cab”} \checkmark$
- Tìm thấy tại vị trí 2

Rabin-Karp: xử lý collision (va chạm hash)

Vấn đề: Hash bằng nhau chưa chắc chuỗi giống nhau!

Ví dụ collision:

$\text{hash}(\text{"abc"}) = \text{hash}(\text{"xyz"}) = 42$ (giả sử)

→ Nếu chỉ so hash → kết luận sai!

Giải pháp trong Rabin-Karp:

1. So sánh hash trước (nhanh - $O(1)$).
2. NẾU hash bằng nhau → so sánh từng ký tự để confirm ($O(m)$).
3. Chỉ khi hash bằng mới so ký tự → tiết kiệm thời gian.

So sánh brute force vs rolling hash

Brute force:

- Mỗi cửa sổ: $so\ m\ ký\ tự$.
- Tổng: $O(n * m)$.

Rolling hash:

- Mỗi cửa sổ: update hash $O(1)$, hiếm khi phải compare m ký tự.
- Trung bình: $O(n)$.

Khi pattern dài, text dài (tìm chuỗi trong file log lớn) $\rightarrow$ rolling hash hiệu quả hơn.

Bài tổng hợp 1: group anagrams

Đề bài:

Cho list string, nhóm lại các từ là anagram của nhau.

Ví dụ: ["eat", "tea", "tan", "ate", "nat", "bat"] →

[["eat", "tea", "ate"], ["tan", "nat"], ["bat"]].

Ý: anagram = cùng số lượng từng chữ cái.

Hashing: Dùng “chữ ký” làm key trong dict.

Cách 1: sort làm key

sorted(w):

- "eat","tea","ate" → "aet".
- "tan","nat" → "ant".

Key giống nhau → vào chung group.

Độ phức tạp: $O(n * k \log k)$, k = độ dài từ.

```
from collections import defaultdict

def group_anagrams(words):
    groups = defaultdict(list)

    for w in words:
        key = ''.join(sorted(w)) # chữ ký
        groups[key].append(w)

    return list(groups.values())

words = ["eat", "tea", "tan", "ate", "nat", "bat"]
print(group_anagrams(words))
```

Cách 2: frequency làm key

Ý tưởng:

- Lưu tần suất 26 chữ cái (a..z).
- Anagram có cùng frequency.

```
from collections import defaultdict

def group_anagrams_freq(words):
    groups = defaultdict(list)

    for w in words:
        count = [0] * 26
        for ch in w:
            count[ord(ch) - ord('a')] += 1
        key = tuple(count) # dùng tuple làm key hash được
        groups[key].append(w)

    return list(groups.values())
```

Không phải sort từng từ → nhanh hơn khi k lớn.

Bài tổng hợp 2: longest consecutive sequence

Đề bài:

- Cho mảng nums (không sort sẵn).
- Tìm độ dài dãy các số liên tiếp dài nhất.

Yêu cầu: $O(n)$.

Ý tưởng hash set cho longest consecutive

Dùng set để lookup $O(1)$:

- B1: cho tất cả phần tử vào set.
- B2: với mỗi x , chỉ bắt đầu đếm khi x là “đầu dãy”: $x-1$ không nằm trong set.
- B3: từ x , đếm $x+1, x+2, \dots$ cho đến khi số tiếp theo không có trong set.

Mỗi phần tử xử lý tối đa một lần $\rightarrow O(n)$.

Code longest consecutive

```
def longest_consecutive(nums):  
    num_set = set(nums)  
    best = 0  
  
    for x in num_set:  
        # chỉ bắt đầu nếu x là đầu dãy  
        if x - 1 not in num_set:  
            length = 1  
            cur = x  
            while cur + 1 in num_set:  
                cur += 1  
                length += 1  
            best = max(best, length)  
  
    return best  
  
print(longest_consecutive([100, 4, 200, 1, 3, 2])) # 4
```

Key: dùng set để kiểm tra tồn tại $O(1)$.

Bài tổng hợp 3: subarray sum = k

Đề bài:

- Cho nums, integer k.
- Đếm số subarray liên tiếp có tổng = k.

Brute force:

- Thử mọi cặp (i, j), cộng từ i đến j $\rightarrow O(n^2)$.

Dùng hash: prefix sum + dict $\rightarrow O(n)$.

Ý tưởng prefix sum + hash map

Gọi $\text{prefix}[i]$ = tổng từ 0..i.

Tổng subarray (l..r) = $\text{prefix}[r] - \text{prefix}[l-1]$.

Muốn $\text{prefix}[r] - \text{prefix}[l-1] = k \rightarrow$

$\text{prefix}[l-1] = \text{prefix}[r] - k$.

Trong lúc duyệt r từ trái sang phải:

- Lưu số lần mỗi prefix đã xuất hiện vào dict.
- Ở vị trí r, cần tìm xem đã từng có $\text{prefix} = \text{current_sum} - k$ hay chưa.

Code subarray sum = k

```
from collections import defaultdict

def subarray_sum(nums, k):
    prefix_count = defaultdict(int)
    prefix_count[0] = 1 # prefix sum = 0 xuất hiện 1 lần

    current_sum = 0
    result = 0

    for x in nums:
        current_sum += x
        # Tìm prefix[l-1] = current_sum - k
        if current_sum - k in prefix_count:
            result += prefix_count[current_sum - k]
        prefix_count[current_sum] += 1

    return result

print(subarray_sum([1,1,1], 2)) # 2
```

Mỗi bước $O(1)$ truy cập dict $\rightarrow$ tổng $O(n)$.



Kết thúc